

Report on Binary classification – BDP Model Gate QA.

Objective

The goal was to design and implement a reproducible machine learning pipeline that:

1. Generates synthetic datasets for multiple domains (e.g., churn, credit scoring, fraud).
2. Trains models in parallel with deterministic train/validation/test splits.
3. Persists models for reuse and reproducibility.
4. Audits models using BDP Model Gate to assess fairness, performance, compliance, and security.
5. Validates whether the package performs as documented in the official Binary Classification – BDP Model Gate guide.

Methodology (How it was achieved)

The following components with their individual roles were written.

1. Dataset Generation: Created synthetic datasets with aligned features (X), targets (y), and protected attributes (protected_df).
2. Splitting: Implemented train_test_validation_split to ensure deterministic row alignment across all components.
3. Parallel Training: Used ThreadPoolExecutor to train logistic regression models on multiple datasets simultaneously.
4. Model Persistence: Saved trained models with joblib for later evaluation.
5. Model Reloading: Built utilities to reload models and re-evaluate them on test sets without retraining.
6. Fairness Auditing: Integrated StructuredGateContext with BDP Model Gate to run validation, fairness, performance, compliance, and security checks.
7. Reporting: Automated report generation and built pretty-print utilities to summarize results.

Results

1. Training: Models trained successfully across datasets, with validation/test accuracies logged.
2. Persistence: Models were saved and reloaded consistently, confirming reproducibility.
3. Auditing:
 - i. Many datasets initially failed fairness checks due to row alignment issues (protected_df vs. X mismatch).
 - ii. After fixes, fairness checks ran correctly when protected attributes were aligned.
 - iii. Performance checks flagged weak models (ROC AUC < 0.8) as PERFORMANCE_RISK, which matches package expectations.
 - iv. Security checks (e.g., adversarial robustness, PII leakage) passed, confirming no vulnerabilities.
 - v. Fairness checks were marked *NOT_APPLICABLE* when no protected attributes were supplied consistent with documentation.
4. Verification: The package behaved exactly as described in the official docs enforcing alignment, skipping inapplicable checks, and producing structured diagnostics.

Key Learnings

1. Deterministic splits are critical for reproducibility and fairness auditing.
2. Protected attributes must be row-aligned with features to avoid false errors.
3. Probability outputs (predict_proba) are required for calibration checks.
4. The package enforces strict contracts it will not produce misleading metrics if inputs are misaligned or missing.
5. **Purpose of the package:** The BDP Model Gate framework is meant to run structured validation, fairness, performance, compliance, and security checks on machine learning models. It wraps a given model and dataset into a StructuredGateContext and then executes a battery of diagnostics.
6. What was observed in reports:

- i. For many datasets, errors like *"context.protected_df has 800 rows, but context.X has 600 rows — they must be row-aligned"* show that the package is enforcing its contract: inputs must be aligned. That's an expected behavior i.e it's catching mis-splits rather than silently producing misleading fairness metrics.
 - ii. For datasets where inputs were aligned, it was observed that full GateReport objects had checks across validation, fairness, performance, compliance, and security. For example, in the churn and purchase reports, detailed results like target leakage, split overlap, feature contract, performance thresholds, and adversarial robustness were outputted. That matches the documented functionality.
 - iii. The fairness checks (e.g., disparate impact, equalized odds) were marked *NOT_APPLICABLE* when no protected_df was supplied. Again, that's the correct behavior as the package doesn't fabricate fairness metrics if protected attributes aren't present.
 - iv. Performance checks flagged roc_auc below threshold when models were weak, which is exactly what the package promises: it enforces minimum performance standards.
7. Verification against docs: The documentation page explains that Model Gate is supposed to:
- i. Validate dataset splits and feature contracts.
 - ii. Audit fairness when protected attributes are available.
 - iii. Check performance thresholds like ROC AUC.
 - iv. Run compliance and security checks (e.g., monotonicity, adversarial robustness, PII leakage). What was observed in the resulting reports matche this specification without fail, and thus the package is doing exactly what it claims.

Future tasks

Creation of more intentionally flawed synthetic datasets to further test the packages ability to flag and catch errors such as disproportionate or skewed distributions, bias, pii leakage, model card injection, amongst other.

Conclusion

Based on both the official documentation and the reports generated, it is observed that the package is behaving as intended. It enforces alignment, skips fairness checks when inputs are missing, and produces structured diagnostics across validation, fairness, performance, compliance, and security.